

虚拟到现实部署与对比分析

——聚焦数字孪生训练、持续数字链路与现实机器狗落地

专项技术报告

面向仿真训练、真机部署、对比分析与闭环优化

二〇二六年四月

摘要

本文聚焦情景剧场演绎系统中“从虚拟环境到真实机器狗”的核心部署链路，重点讨论数字孪生环境构建、示教数据采集、基于 PPO 的策略学习、持续数字链路下的运行时控制、真实世界对齐以及仿真与现实之间的差异分析。与面向完整系统的总技术报告不同，本文有意弱化前后端内容生产和管理平台的细节，只把它们视作高层剧本语义的来源；全文主体围绕“仿真如何训练”“运行时如何解释”“真实机器人如何执行”以及“现实反馈如何反向修正仿真策略”四个问题展开。

本文首先给出虚拟到现实部署的总体闭环框架，说明为什么本项目不把部署理解为一次性模型导出，而是理解为“虚拟训练—运行时推理—真机执行—真实回流—再次优化”的持续迭代过程。随后，文章详细介绍 Unity 数字孪生环境中预设动画与模板动作的采样方法，说明如何把表演性动作演示转化为可用于策略学习的状态—动作时序数据；在真实侧，则重点分析白名单动作采集、世界相机位姿对齐和真实误差建模，说明为什么真实世界必须作为策略修正依据，而不能仅仅被当作最终展示舞台。

在控制与部署部分，本文统一采用“持续数字链路”这一表述来替代传统请求式接口描述，强调 Unity 运行时和嵌入式控制器之间需要一条长连接、连续更新、双向反馈的数据通道，从而支撑高频角度下发、状态回传、确认机制、超时保护与风险裁决。在对比分析层面，本文从状态可观测性、动作平滑性、误差来源、迭代成本、调试便利性和安全可控性等多个维度，对虚拟环境与现实环境进行了系统比较，指出二者既非替代关系，也不是简单的先后关系，而是职责互补、信息互补、价值互补的双域系统。

最终，本文形成一份面向后续答辩、复盘与二次开发的聚焦型技术文档。它既可作为完整系统报告中“虚拟到现实部署”章节的独立扩展，也可作为训练与部署团队的专项归档材料。

关键词：数字孪生；虚拟到现实；PPO；Behavior Cloning；持续数字链路；四舵机机器狗；Unity 运行时；真实世界对齐

目录

摘 要.....	2
1 研究背景与写作范围.....	4
2 虚拟到现实部署的总体闭环.....	4
3 Unity 数字孪生环境构建与虚拟侧采集.....	5
4 真实世界对齐：白名单动作、位姿回读与现实约束建模.....	6
5 策略模型的输入、输出与训练逻辑.....	7
6 运行时部署：从策略输出到持续数字链路下发.....	8
7 嵌入式执行：ServoProfile、平滑控制与安全保护.....	9
8 虚拟与现实的多维对比分析.....	10
9 现实部署中的常见偏差与修正策略.....	11
10 面向后续迭代的闭环优化路线.....	12
11 结论.....	12
附录 A 虚拟与现实对比摘要表.....	13
附录 B 持续数字链路中的关键数据帧.....	14
附录 C 建议持续跟踪的部署指标.....	14

1 研究背景与写作范围

在情景剧场演绎系统中，真正困难的部分从来都不是让一只机器狗“动一下”，而是让它在高层剧本语义约束下，以连续、可解释、可复现的方式完成完整表演。对于一套同时包含剧本生成、仿真训练与实体控制的系统来说，如果只强调上层软件生成能力，就容易把机器人执行过程理解成一个黑箱；如果只强调硬件控制，又会失去项目最大的特色，即高层叙事语义能够逐步被翻译为低层舵机控制。因此，当我们把视角从完整系统技术总报告进一步收缩到“虚拟到现实部署”主题时，关注点就会自然集中到三件事情上：虚拟环境如何采集与训练，运行时如何解释与推理，真实机器狗如何接收并执行控制流。

本文件正是在这样的背景下形成的。它不再详细展开前后端工作台、内容管理界面、多智能体分工、数据库组织等上层软件细节，而只把软件系统视作“高层剧本条件的提供者”。换句话说，本文默认上层已经能够产出动作标签、镜头阶段、角色上下文和执行时序；在此前提下，我们要解决的问题是：如何在 Unity 中构建足够可用的数字孪生环境，如何采集预设动画和模板动作，如何利用示教与强化学习获得可部署策略，如何通过持续数字链路把策略输出送到真实机器人，以及如何把真实执行偏差重新带回训练与优化阶段。

从工程意义上看，这种写法也更加贴合系统落地环节的真实工作分工。因为在实际联调过程中，前后端团队、仿真训练团队和嵌入式团队并不是同时盯着同一组问题。上层更关心剧本结构是否完整，中层更关心状态空间与策略是否合理，下层更关心控制是否稳定、链路是否可靠和真机是否安全。本文试图为中下层链路提供一份统一口径的专项技术报告，让团队在讨论虚拟到现实部署时，能围绕同一套术语、同一套链路和同一套约束开展协作。

2 虚拟到现实部署的总体闭环

在很多项目里，所谓“部署”往往被简化为把模型文件导出到某个运行环境中，再让设备按模型输出去执行。但对于本项目而言，这样的理解显然不够。因为我们的目标

并不是把一段离线动作轨迹复现到真实机器狗上，而是让机器狗在接收到剧本语义和环境状态后，按照控制周期持续生成并执行新的舵机角度流。这意味着，部署不是单点事件，而是一条持续工作的闭环链路。我们把这条链路概括为：虚拟采集、策略预热、强化优化、运行时推理、真实执行、真实回流、再训练修正。

闭环的第一段发生在 Unity 数字孪生环境中。这里既承担示教数据采集，也承担 PPO 所需的大量环境交互。由于仿真环境可以重复、可并行、风险低，因此它特别适合生成大规模训练样本，并为策略提供充分的试错空间。闭环的第二段发生在 Unity 运行时。模型训练完成后并不是直接被烧录到嵌入式侧，而是先进入 Unity 本地执行器，由它在真实剧本输入和世界位姿输入的条件下，按固定控制频率生成下一时刻的关节控制目标。闭环的第三段发生在真实机器人。这里真正暴露了舵机迟滞、装配误差、供电波动、接地摩擦与链路延迟等仿真中难以完全还原的约束。

关键之处在于，这三段不是串完一次就结束，而是彼此反哺。真实机器人执行中产生的轨迹误差、动作失真、风险触发和模板偏差会重新进入分析环节，进而修正状态定义、奖励函数、白名单动作库甚至 ServoProfile 参数。这也是我们始终强调“虚拟到现实是一条持续迭代闭环”的原因。只有把真实世界重新拉回训练视野，系统才能从“仿真中看起来合理”逐步走向“现实中可以长期稳定运行”。

为了让这条闭环在工程上真正可执行，我们还明确划分了各层职责：虚拟侧负责产生高覆盖度训练数据与试错空间；运行时负责把模型能力变成按周期工作的控制流；嵌入式侧负责安全、平滑、可确认地执行这些控制流；真实回流侧负责用可观测指标将现实误差结构化。职责清晰之后，团队才不会把所有问题都推给“模型不好”或“硬件不稳”，而是能定位到具体一层去修正。

3 Unity 数字孪生环境构建与虚拟侧采集

如果说虚拟到现实部署的第一步是什么，那么答案绝不是“先训练一个 PPO 模型”，而是先建立一个足够可用、足够一致、足够能采样的数字孪生环境。对本项目而言，数字孪生环境不是单纯摆出一只四足模型，更不是仅仅制作几段漂亮动画，而是要在环境状态、关节约束、动作节奏和采样接口上尽可能向真实机器狗靠拢。只有这样，后续得到的示教数据和强化学习结果才具有迁移意义。

在环境构建中，我们首先统一了机器狗的低自由度结构表达方式。由于当前实体平台每条腿只有一个主要控制自由度，因此虚拟模型也必须围绕这一硬件事实来建模。这样做虽然牺牲了复杂动作表现力，但换来了动作空间、策略输出与真实执行之间的一致性。其次，我们把训练环境中的地面、边界、障碍物和目标方向都显式纳入状态定义，使仿真阶段就能看到与现实部署相关的安全信息，而不是把环境仅仅视作“空旷舞台”。

在预设动画采集阶段，我们把每一段动画都看作带标签的专家轨迹。采集并不只截取最终角度，而是按固定控制频率记录整个时间序列中的动作标签、phase、剩余时长、位置、朝向、速度、四个控制关节的目标值以及环境风险变量。这样做的价值非常大：它把原本只适合展示的动画，转化为了可以直接进入 Behavior Cloning 和 PPO 初始化阶段的数据样本。更重要的是，数据组织形式已经尽可能贴近真实部署时的输入输出关系，因此后续策略学习不会与实际控制链路脱节。

我们还特别强调动画到舵机空间的映射。动画系统输出的是骨骼姿态，而控制系统需要的是四个关节的连续数值，所以中间必须建立稳定的一一映射。为此，项目通过统一的关节映射表和 ServoProfile 范围配置，把虚拟侧的关节角归一化到统一动作空间，再把它们与真实硬件允许的最小角、最大角、中位角和单步最大变化量绑定起来。这样，策略学习面对的始终是标准化动作空间，而不是某台样机偶然的装配姿态。

从数据价值上看，虚拟侧采集不仅提供“能学会动作”的示教样本，也提供“能和真实侧对齐”的结构化中间表示。正因为预设动画采得足够细，后续我们才能将策略理解为条件时序控制器，而不是简单动作分类器。

4 真实世界对齐：白名单动作、位姿回读与现实约束建模

仿真环境再完善，也不会直接等于真实世界。真实世界的困难往往不是某个数学公式，而是大量非理想因素的叠加：舵机响应有迟滞，机械装配有偏差，机身重心会漂移，地面摩擦与材料接触不一致，相机定位也可能存在时延与噪声。如果忽略这些现实约束，训练出来的策略很容易在仿真里走得很好看，到了真实机器狗上却出现动作抖动、偏航甚至碰撞。正因如此，本项目在部署体系中把“真实世界对齐”放到和虚拟训练同样重要的位置。

现实对齐的第一项工作，是建立白名单动作模板库。所谓白名单动作，指的是已经在真实硬件上被反复验证、具有较高稳定性和可复现性的基础动作，例如站立、前进、左转、右转、后退、打招呼和回中姿态等。这些动作并不是对学习策略的否定，反而构成了策略校正与系统保底的重要基础。一方面，它们提供了真实世界中可靠的动作参考轨迹，使团队能够把策略输出与成熟模板进行对照；另一方面，当模型输出存在明显风险或链路异常时，系统也可以临时退回白名单动作，保证现场展示不会彻底失控。

现实对齐的第二项工作，是利用世界相机或等效定位手段获取真实机器狗的结构化位姿信息。我们并不把世界相机用于端到端感知，而是把它看作外部定位参考源，主要输出位置、朝向、跟踪置信度和与边界障碍物相关的几何量。这样做的好处在于，虚拟侧和现实侧的状态空间可以更容易对齐：策略真正关心的是当前位置、当前朝向和距离约束，而不是原始像素。通过把真实位姿投影回 Unity 场景，运行时就能够在推理阶段使用与训练中近似一致的状态定义。

现实对齐的第三项工作，是把真机执行结果系统化记录下来。项目不仅记录控制目标，还记录执行后状态、角度到位偏差、动作完成情况、风险触发情况和定位轨迹误差。只有这些数据被保留下来，团队才有可能做后续分析，例如判断某个动作为什么在现实中偏航、某一类 **phase** 为什么容易失稳、某一组舵机组合为什么在特定地面上更容易抖动。与其把真实部署当成“最后一关”，不如把它看成持续产生高价值训练反馈的试验场。

5 策略模型的输入、输出与训练逻辑

在模型设计上，本项目从一开始就避免将策略做成“输入一个标签，吐出一整段轨迹”的离线动作生成器。因为真实机器人执行始终处在变化环境中，任何一次定位波动、硬件延迟或意外接触，都可能使原定轨迹在下一时刻失效。因此，我们训练的是一个逐时刻工作的条件策略：它在每一个控制周期只负责根据当前状态和当前上下文，输出下一时刻四个控制关节的目标值或增量值。

模型输入主要由四部分组成。第一部分是当前世界状态，包括位置、朝向、线速度、角速度以及由环境几何计算得到的障碍距离与边界安全裕度。第二部分是当前动作上

下文，包括当前动作标签、当前动作阶段 **phase**、剩余执行时长和必要的风格描述。第三部分是历史控制状态，例如上一时刻和最近若干时刻的关节角、角度差分和风险历史，以帮助策略维持动作连续性。第四部分是机体内部状态，例如躯干姿态、倾角或能反映当前稳定性的辅助变量。这样的输入设计使策略既知道“我现在应该做什么”，也知道“我当前处在什么环境与动力状态中”。

模型输出则尽量保持与真实控制链路一致。工程上，我们更偏向输出关节角增量，而不是每一步直接输出绝对角度。原因在于，增量形式更容易与平滑约束、速率限制和硬件安全保护结合，也更符合低自由度四舵机系统在实际运行中的过渡控制需求。无论输出是绝对值还是增量值，最终都要经过 **ServoProfile** 和底层保护逻辑映射为真实可执行角度，再进一步转为脉宽与占空比。

在训练流程上，我们坚持“示教预热 + PPO 优化 + 真实回流修正”的组合。示教阶段使用虚拟侧采集到的预设动画和模板动作，让模型先掌握基本动作轮廓，尽快落入合理动作流形附近；PPO 阶段则通过任务完成奖励、平稳性奖励、节奏一致性奖励和安全惩罚，推动策略在交互中学会更稳、更自然、更符合环境约束的控制方式；真实回流阶段再利用白名单模板和真机日志对模型做监督微调、残差拟合或参数校正。整个过程中，策略不是孤立成长的，而是在“虚拟探索”和“现实校正”双重驱动下逐步成熟。

这种训练逻辑的核心价值，在于它避免了两个极端：一是不让模型完全依赖人工模板，失去在扰动环境中的自适应能力；二是不让模型从零开始盲目探索，浪费样本并造成大量无效动作。它既尊重已有模板经验，也给策略学习留出了优化空间。

6 运行时部署：从策略输出到持续数字链路下发

部署阶段的关键不是“模型文件放在哪里”，而是“模型输出如何在运行时变成真实可执行控制”。在本项目中，模型并不直接部署到嵌入式芯片内部，而是首先部署到 **Unity** 运行时中。**Unity** 作为中间层有两个重要优势：一是它能够同时接收上层剧本状态和外部位姿信息，方便构造完整输入；二是它能够在同一运行环境中完成推理、安全裁决、节奏控制与状态缓存，而不必把这些复杂逻辑都压到嵌入式端。

在一个典型控制周期内，Unity 运行时会先读取当前剧本标签、当前镜头阶段和世界位姿，再把历史状态窗口与环境约束一起组织成模型输入。推理器输出四个关节的下一步目标后，并不会立刻原样发送给机器人，而是先经过安全裁决器。安全裁决器会检查角度是否越界、相邻两帧变化是否过大、当前方向是否可能导致撞墙以及是否触发了其他风险约束。如果结果不安全，系统可以做限幅、降速、暂停推进，甚至切回白名单动作。只有通过这一层裁决后，控制值才会进入持续数字链路。

这里特别需要强调的是，本项目已经统一用“持续数字链路”来描述运行时与机器人之间的通信机制，而不再使用请求式接口作为主控制路径。持续数字链路的本质，是在 Unity 与嵌入式控制器之间建立一条持续存在、双向工作的数据通道。Unity 按固定频率持续下发 `ControlFrame`，嵌入式侧按固定频率持续回传 `StateFrame`，二者之间还存在 `HeartbeatFrame`、`SyncFrame` 和 `EventFrame` 等管理型数据帧。与低频调用式通信相比，这种方式更适合高频连续控制，因为它保证了消息顺序、状态确认与链路保活。

持续数字链路在工程上的价值不只是更“快”，更在于更“稳”和更“可控”。由于系统可以带着序号发送控制帧，并在状态帧中回传最近确认序号，Unity 运行时就能明确区分“已经发出”“已经收到”“正在执行”和“基本到位”几个阶段。这种显式确认机制对于强化学习部署尤其重要，因为模型输出本质上是连续控制量，只有当上层知道这些量在现实中是否真的落到了机器狗身上，闭环才算成立。

7 嵌入式执行：ServoProfile、平滑控制与安全保护

当控制值通过持续数字链路到达嵌入式侧后，真正的硬件执行才开始。需要注意的是，模型和 Unity 输出的仍然只是统一动作空间中的关节目标，而舵机底层真正识别的是脉宽与定时器输出。因此，嵌入式侧必须承担起动作中间层到硬件驱动层的转换责任。这一转换由 `ServoProfile` 管理：它记录每个关节的零位、中位角、最小角、最大角、死区和单步最大变化量，并据此把上层输出映射为真实脉宽和占空比。

保留 `ServoProfile` 的意义非常大。如果没有这层中间抽象，策略就会直接依赖某一台机器人当前的机械装配与校准状态，一旦更换样机、重新安装舵机或调整连接杆长度，模型就可能整体失效。而有了 `ServoProfile`，模型始终面对统一动作空间，真实硬件差异则通过配置参数被吸收。这样不仅便于维护，也更适合后续升级硬件结构。

在执行逻辑上，嵌入式侧不会收到角度后瞬间强拉到目标，而是通过固定周期控制任务做渐进式推进。网络接收任务负责解析控制帧并更新目标值；控制执行任务以固定频率运行，对目标值进行限幅、插值和平滑处理后再映射为 PWM 输出；状态回传任务则持续把当前角度、最近确认序号、模式状态和异常标志打包发回上层。这样的任务分离方式避免了网络抖动直接影响舵机执行节奏，也避免了执行阻塞影响状态上报。

安全保护贯穿整个嵌入式执行过程。系统会对每一个输入角度做格式检查和范围裁剪，对相邻帧变化幅度做速率限制，并在长时间未收到新控制或心跳时启动超时失联保护，让机器人缓慢回到中立姿态。对于现场演示来说，这些保护并不只是为了“避免烧机”，更是为了维持整套系统的可信性：观众看到的应当是一只可在可控边界内表演的机器狗，而不是一台随时可能因链路波动出现剧烈抽动的装置。

8 虚拟与现实的多维对比分析

从开发经验看，虚拟环境和现实环境各自承担了不可替代的角色。虚拟环境最大的优势是样本成本低、失败风险低、可重复性高且易于并行。研究者可以在 Unity 中快速改变地面参数、障碍布局、初始位姿和动作条件，也可以同时开出多个训练环境，显著提高 PPO 的数据吞吐。这使得策略能够在较短时间内经历大量尝试，获得更充分的探索机会。对算法而言，虚拟环境是最主要的学习空间。

现实环境的优势则截然不同。它并不提供海量廉价样本，却能够暴露那些对最终落地最关键的误差来源：舵机迟滞、机械松动、供电波动、地面差异、定位噪声、装配偏心和控制链路时延。很多在虚拟环境里看似稳定的动作，到了现实中会因为这些误差而表现出明显偏差。也正是现实环境，让团队真正看见策略的部署边界与保底需求。对系统而言，现实环境是最主要的校正空间。

从状态可观测性的角度比较，虚拟环境几乎可以无成本获取位置、速度、朝向、碰撞与边界距离等一切需要的变量；现实环境则必须依赖外部定位、回读角度、日志聚合和置信度评估来重建这些状态，因此可观测性更贵也更不稳定。正因为如此，现实部署时更需要状态筛选、平滑和冗余保护。

从动作平滑性的角度比较，虚拟环境中的动作输出通常更加理想，因为物理参数已知且执行器没有真实硬件老化、齿隙和弹性变形；现实环境中的动作平滑性则取决于舵机质量、插值策略、供电条件与机械结构，因此更依赖平滑器、速率限制和模板保底。

从迭代成本看，虚拟环境允许快速试错，适合改状态空间、改奖励函数、改动作定义；现实环境则更适合少量高价值验证，因为每次真机试验都要消耗时间、设备寿命和现场风险预算。综合来看，二者最合理的分工不是谁替代谁，而是虚拟负责广覆盖探索，现实负责高可信校正。

9 现实部署中的常见偏差与修正策略

在实际部署中，常见偏差往往并不是“模型完全不会动”，而是一些更隐蔽但持续影响表演质量的问题。例如前进行为可能总体正确，但步幅比仿真更小；转向行为可能方向没错，但转角积累误差较大；打招呼等表演动作可能视觉上成立，但相邻阶段衔接不够自然；某些动作在地面更滑时容易出现抖动或偏航。对于这些问题，如果只靠肉眼判断，很容易停留在经验性修改层面。

因此，项目在修正策略上强调指标化与模板化。首先，通过角度到位误差、位姿漂移、阶段完成率、风险触发次数和平均推理周期等指标，把偏差结构化。其次，通过白名单模板和真实轨迹库，把“正确示范”固化下来，便于将策略输出与高可信模板进行比较。最后，再根据偏差性质选择修正方式：若问题主要来自策略相位不稳，可调整 `phase` 编码和奖励节奏；若问题主要来自硬件边界，可修改 `ServoProfile` 或限速参数；若问题主要来自环境差异，可增加域随机化或现实样本回流。

在很多时候，最有效的修正并不是直接重训整套模型，而是在不破坏主策略结构的前提下增加一层残差补偿或运行时修正。比如，若真实转向普遍偏小，可以在同类动作条件下叠加小幅增益；若真实执行在某个区间内总有迟滞，可以通过控制任务调整单步逼近逻辑。也就是说，虚拟到现实的迁移不必只依赖“大模型再来一次”，很多时候更适合通过小步、可解释、可验证的工程修正逐步逼近。

对于现场演示系统而言，另一个极其重要的修正原则是“优先保证稳定，再追求丰富表现”。也就是说，当高难动作尚未充分可靠时，应先让机器人在核心动作和核心节

奏上稳定发挥，再逐步把更复杂的动作风格引入系统。剧场演绎的可信感往往来自连续稳定，而不是单次炫技。

10 面向后续迭代的闭环优化路线

从长期演进角度看，虚拟到现实部署绝不应该被视为“做完一次就结束”的冲刺任务，而应视作系统生命周期中的常规能力。当前系统已经具备了从虚拟采集到真机执行再到真实回流的基本闭环，但要进一步提高稳定性与表现力，后续仍有几条明确的优化路线。

第一条路线是继续丰富真实世界样本，但不是盲目堆量，而是有针对性地补充那些最能暴露现实误差的片段，例如边界附近转向、连续阶段切换、长时间前进后的姿态漂移和不同材质地面上的稳定性变化。这些样本比简单重复演示更具修正价值。第二条路线是继续完善域随机化，使仿真环境更主动地模拟现实中最常见的偏差来源。第三条路线是将白名单模板与学习策略结合得更紧密，例如发展模板引导下的残差学习，而不是让二者平行存在。

第四条路线是增强持续数字链路上的运行时可观测性。未来可进一步把链路延迟、帧丢失率、确认滞后、执行周期抖动等指标纳入监控，使团队能够更加精确地区分“策略问题”“链路问题”和“执行问题”。第五条路线是考虑在保持当前低自由度系统优势的前提下，逐步扩展更丰富的动作标签或更细粒度的节奏条件，使机器人在剧场场景中的表达力进一步提升。

归根结底，虚拟到现实部署的成熟度不取决于某一次演示是否成功，而取决于系统是否形成了稳定的知识回流机制：每一次失败、每一次偏差、每一次保底切换，最终都能够转化为下一轮训练、下一轮调参或下一轮结构改进的依据。只有当系统能够持续吸收这些经验，它才真正从“演示原型”成长为“可迭代平台”。

11 结论

本文围绕情景剧场演绎系统中最关键也最容易被低估的一条链路——虚拟到现实部署——进行了聚焦分析。与完整技术总报告不同，本文有意将前后端内容生产细节收缩为背景条件，把主要篇幅集中在数字孪生环境构建、虚拟侧示教采集、真实世界对

齐、条件策略训练、持续数字链路控制、嵌入式执行以及虚拟与现实的多维对比上。通过这一聚焦，我们可以更清晰地看到：系统的真正难点并不是单纯把模型导出来，而是让模型、运行时、通信链路和真实硬件在统一状态定义下长期协同工作。

从方法上看，本项目已经形成了一条较为完整的闭环路径：在 Unity 中通过预设动画和模板动作采集结构化示教数据，以 Behavior Cloning 预热策略，再以 PPO 进行强化优化；在运行时中利用当前剧本标签、phase、世界位姿和历史状态构造输入，持续输出下一时刻关节控制；通过持续数字链路把控制帧稳定下发到嵌入式端，并借助状态帧和确认机制建立双向闭环；在真实执行中以白名单动作、ServoProfile、平滑控制和超时保护维持安全边界；最后把现实中的位姿偏差、角度误差和风险事件重新纳入分析与修正。

更重要的是，本文所强调的“虚拟负责广覆盖训练，现实负责高可信校正”的分工，为后续系统迭代提供了清晰方向。未来无论是增加动作标签、升级舵机、引入更多传感器，还是优化策略模型，都可以继续沿着这条闭环展开，而不必推翻现有架构。也就是说，虚拟到现实部署在本项目中并不是一个附属环节，而是整套具身表演系统能否真正落地的主干。

附录 A 虚拟与现实对比摘要表

下表从多个关键维度概括虚拟侧与现实侧在本项目中的职责差异与协同关系，便于后续团队在复盘和答辩中快速引用。

维度	虚拟侧特点	现实侧特点	在本项目中的作用
样本获取	成本低、可并行、可反复失败	成本高、次数有限、需控制风险	形成“广覆盖探索 + 高价值验证”的组合
状态可观测性	位置、速度、碰撞等变量可直接获取	依赖相机、回读和日志重建	决定现实部署更依赖冗余保护与平滑
动作质量	轨迹理想、可控、噪声可配置	受舵机与机械误差影响更大	促使系统保留白名单模板与 ServoProfile
调试效率	快速试错，适合改奖励与状态空间	调试慢，但能暴露真实问题	形成“先仿真验证，再真机压测”的节奏
安全风险	几乎无硬件风险	存在碰撞、抖动和过载风险	要求运行时裁决与超时保护
核心价值	训练与探索主场	校正与部署主场	二者共同构成 Sim-to-

			Real 闭环
--	--	--	---------

附录 B 持续数字链路中的关键数据帧

为保持文档口径一致，本文不再采用请求式接口描述，而用持续数字链路中的关键数据帧来说明运行时和真实机器人之间的交互方式。

数据帧类型	方向	主要内容	作用
ControlFrame	Unity → Robot	四关节目标值、phase、序号、过渡时长	按控制周期持续下发控制量
StateFrame	Robot → Unity	当前角度、模式、ack_seq、异常标志	回传执行状态并确认最近接收帧
HeartbeatFrame	双向	时间戳、节点编号、链路状态	维持链路活性并检测超时
EventFrame	Robot → Unity	完成、急停、越界、重连等事件编码	上报低频但关键的系统事件
SyncFrame	双向	协议版本、设备信息、模式切换参数	建链初始化与参数同步

附录 C 建议持续跟踪的部署指标

为了让虚拟到现实闭环真正可积累、可复盘，建议对以下指标进行长期记录。

指标名称	说明	所属层
角度到位误差	目标角与回读角之间的偏差统计	嵌入式执行层
阶段完成率	一个镜头或动作阶段是否按预期结束	Unity 运行时层
平均推理周期	单次推理与裁决耗时	Unity 运行时层
链路确认延迟	控制帧发送到 ack 返回的时延	持续数字链路
风险触发次数	越界、急停、保底切换等事件频次	全链路
真实位姿漂移	现实轨迹与预期轨迹之间的偏差	Sim-to-Real 对齐层